

Noções Básicas de Estruturas de Dados

Laboratórios de Informática III

Guião #4

Departamento de Informática
Universidade do Minho

Outubro de 2025

Conteúdo

1	Introdução	2
2	Estruturas de dados comuns	2
2.1	Arrays	2
2.2	Listas ligadas	3
2.3	Stacks	4
2.4	Tabelas de hash	4
2.5	Árvores binárias de pesquisa	5

1 Introdução

Estruturas de dados organizam conjuntos de dados em formatos específicos. Cada um destes formatos apresenta vantagens e desvantagens em relação a operações de inserção, remoção, e pesquisa de dados, custo de memória, entre outros. Assim, no momento de escolher uma estrutura de dados, devem ser tidas em conta as necessidades do nosso caso de uso.

Por exemplo, no caso de querermos implementar uma fila de espera, só precisamos de aceder às posições inicial e final da fila, pelo que a poderemos implementar recorrendo à Deque do `Guiaão-1`. Noutra instância, se precisarmos de inserir dados em posições aleatórias de uma lista, para adicionar um novo elemento de forma ordenada, a estrutura de lista ligada será uma melhor opção.

Em suma, nunca existirá uma estrutura de dados boa para todos os casos e, portanto, devemos olhar para o tipo e frequência das operações que queremos efetuar. O processo de escolha deve começar por responder a questões como:

- Que operações quero suportar?
- Que operações vou efetuar mais frequentemente?
- A minha prioridade é poupar espaço em memória ou efetuar as operações mais rapidamente?

A partir destas respostas podemos rapidamente identificar uma estrutura que se adequa ao nosso problema.

2 Estruturas de dados comuns

Existe um número considerável de estruturas de dados, podendo existir para uma qualquer estrutura um grande número de variações da mesma. Neste guia, apresentamos apenas um conjunto pequeno destas estruturas, que evidenciam comportamentos muito diferentes entre si, e que como tal são aplicadas em contextos diferentes.

Assim, consideramos as seguintes estruturas: arrays, listas ligadas, stacks, tabelas de hash, e árvores binárias. Para cada uma delas, analisamos a complexidade das operações de inserção, remoção, acesso e pesquisa. Abaixo encontram-se alguns dos níveis possíveis de complexidade das operações, por ordem crescente:

- **Constante** - quando a operação envolve o mesmo número de passos, independentemente do número de elementos na estrutura. Por exemplo, para consultar uma posição de um array, o tempo de execução será aproximadamente o mesmo quer o array tenha 10 ou 100 elementos.
- **Logarítmico** - quando o número de passos envolvidos tem uma relação logarítmica com o número de elementos na lista. Por exemplo, numa árvore binária balanceada iremos visitar até $\log_2 N$ nós para encontrar o nó que pretendemos.
- **Linear** - quando o número de passos envolvidos numa operação cresce de forma linear com o tamanho de uma lista. Por exemplo, se num array com 10 elementos, em média, demoramos 10ms para pesquisar um nó, num array 10x maior, com 100 elementos, iremos demorar por volta de 100ms, *i.e.*, 10x10ms.

Outros níveis de complexidade incluem: complexidade polinómica e complexidade exponencial.

2.1 Arrays

Um array é formado por um bloco contíguo de memória, dividido em várias posições, sendo que cada posição do array é capaz de guardar um elemento. Por exemplo, no caso de um array de números decimais, cada posição no array é capaz de guardar um número, como exibido na Figura 1.

```
double x[8];
```

Array x

x[0]	x[1]	x[2]	x[3]	x[4]	x[5]	x[6]	x[7]
16.0	12.0	6.0	8.0	2.5	12.0	14.0	-54.5

Figura 1: Arquitetura de um array
Fonte: cis.temple.edu

O **acesso** a um elemento da lista ocorre em tempo **constante**, já que qualquer elemento pode ser facilmente acessado sabendo-se a sua posição no array. No entanto, a complexidade de **pesquisa** nesta estrutura é **linear**, uma vez que o array tem que ser percorrido até que o elemento pretendido seja encontrado, sendo que essa travessia irá ser linearmente proporcional ao número de elementos no array. A inserção e remoção de valores em posições aleatórias do array incorrem numa penalização quando a posição em questão se encontra no meio dos outros elementos do array. Quando queremos inserir um valor no meio de outros elementos será necessário mover elementos para abrir espaço para o valor que queremos colocar no array. De forma semelhante, ao remover um elemento do meio do array será necessário mover elementos para ocupar o espaço que foi deixado em branco pelo elemento removido. Assim, a **inserção e remoção** de elementos em arrays é de complexidade **linear**.

Em termos de memória, arrays ocupam um espaço fixo, o que significa que, *e.g.*, para um array com 100 posições, vamos estar a ocupar o espaço necessário para 100 elementos mesmo que só tenhamos preenchidas 10 posições. Para além disso, se o array esgotar a sua capacidade e quisermos continuar a inserir elementos teremos que alocar um novo array em memória, com maior capacidade, e copiar o conteúdo do array antigo para o novo. No entanto, se soubermos exatamente de quanto espaço vamos precisar (*i.e.*, não vamos deixar posições vazias) o array é a estrutura que mais eficientemente utiliza o espaço em memória, uma vez que todo o espaço é dedicado a guardar dados, não requerendo elementos extra de manutenção da estrutura, como por exemplo apontadores.

Um array é normalmente utilizado quando temos garantias sobre o número de posições que vamos necessitar. Imaginemos um sensor de medição de corrente que efetua uma medição a cada 10ms e reporta o valor médio, mediano, máximo e mínimo de corrente do último 1s (*i.e.*, 100 medições). Aqui pode ser usado um array para guardar os valores medidos durante um segundo, e depois iterar sobre os mesmos para obter as métricas pretendidas. Uma vez que os elementos de um array estão localizados num espaço contíguo de memória, a iteração sobre os elementos do array é mais eficiente, devido à localidade espacial dos dados. Um array pode também ser útil para representar operadores em equações matemáticas. Estando situados em posições contíguas de memória passa a ser possível aplicar operações de vetorização que permitem fazer várias operações matemáticas em simultâneo (*e.g.* multiplicar os elementos entre dois vetores).

2.2 Listas ligadas

As listas ligadas são compostas por nós interligados através de apontadores. Cada nó é alocado de forma separada, pelo que não é consumido espaço em memória para além dos nós que se encontram atualmente na lista. No entanto, cada nó consome em memória o espaço extra correspondente a um (ou mais) apontadores. A Figura 2 representa a organização geral de uma lista duplamente ligada, em que cada nó aponta para o nó seguinte e anterior ao seu na lista.

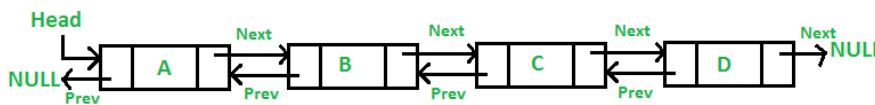


Figura 2: Arquitetura de uma lista duplamente ligada

Fonte: geeksforgeeks

Listas ligadas são úteis em cenários em que elementos são inseridos e removidos frequentemente, não tendo a lista um tamanho fixo nem conhecido. E também em cenários onde elementos são frequentemente inseridos em posições internas da lista, já que não é necessário alterar a posição em memória dos outros elementos. Assim, a lista ligada tem uma complexidade de **inserção e remoção constante** em qualquer posição, uma vez que estas operações irão sempre envolver, no máximo, modificações em 2 elementos já existentes na lista. A complexidade de **pesquisa e acesso** é **linear** para listas ligadas, visto que a lista tem que ser percorrida nó a nó até se chegar à posição ou nó desejado.

Listas ligadas podem ser usadas, por exemplo, para manter o estado sobre páginas visitadas num separador web, permitindo facilmente navegar entre elas; ou para representar filas de reprodução em aplicações de áudio como o Spotify, facilitando a navegação entre as músicas da fila, a adição e remoção de músicas à fila e a alteração da ordenação das músicas na mesma.

2.3 Stacks

Stacks são estruturas de dados semelhantes a listas, mas com funcionalidade mais restrita. Os elementos apenas podem ser inseridos e removidos à cabeça da estrutura, numa configuração LIFO (Last-in-First-Out), *i.e.*, apenas o elemento colocado mais recentemente na Stack pode ser removido, como mostra a Figura 3.

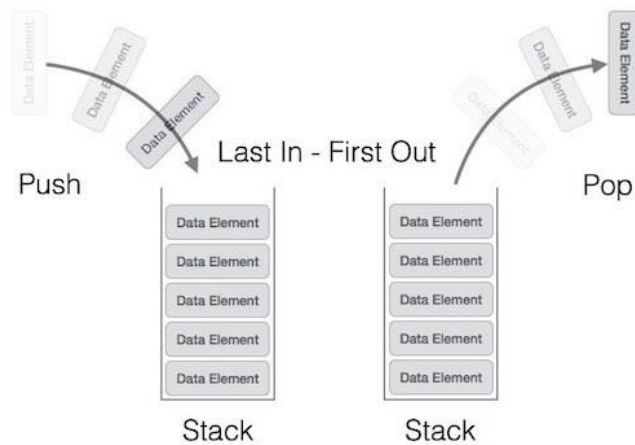


Figura 3: Arquitetura de uma stack
Fonte: tutorialspoint

Na maioria dos casos, as Stacks são implementadas recorrendo a arrays ou listas ligadas. A escolha entre uma ou outra irá depender do caso de uso específico.

As Stacks podem servir cenários como: manter o histórico de modificações efetuadas a um ficheiro de texto, para facilitar operações de `undo` e `redo`; ou rastrear a sequência de funções chamadas num programa, como é o caso da stack de funções de programas C.

2.4 Tabelas de hash

As tabelas de hash são estruturas que guardam pares chave-valor. Cada chave é mapeada para uma posição em memória onde é guardado o par, de tal forma que, dada uma chave, a **pesquisa** na estrutura pelo valor correspondente é feita em tempo **constante**. É possível que duas chaves diferentes sejam mapeadas para a mesma posição de memória. Quando ocorrem colisões deste género é aplicada uma política de resolução de conflitos. A organização geral de uma tabela de hash segue a forma da Figura 4.

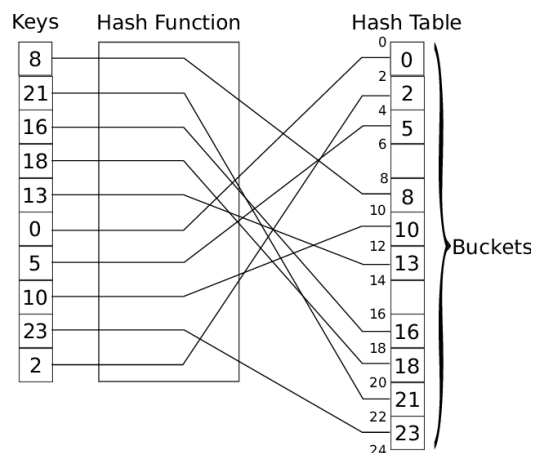


Figura 4: Arquitetura de uma tabela de hash
Fonte: researchgate

Apesar da **inserção e remoção** de valores ser efetuada em tempo **constante**, a complexidade destas operações envolve mais passos do que, por exemplo, um array, uma vez que o mapeamento de uma chave para a posição correspondente do par em memória envolve o cálculo de uma função de hash. Assim, as tabelas de hash são maioritariamente aplicadas em situações que requeiram tanto a consulta rápida de valores a partir da sua chave, como tempos de inserção relativamente rápidos.

A implementação de uma tabela de hash normalmente recorre ao uso de uma estrutura de array para guardar os pares chave-valor, podendo ainda usar listas ligadas para efeitos de gestão de colisões.

Alguns exemplos de cenários onde tabelas de hash podem ser aplicadas incluem servidores de DNS, onde é feita a consulta de endereços IP a partir de nomes de domínio (e.g., chave - `www.google.com`, valor - `142.250.185.14`); e em serviços de encurtamento de URLs, onde os URLs encurtados são mapeados para os seus URLs originais (e.g., chave - `https://tinyurl.com/yfp9jchm`, valor - `https://www.google.com/search?q=how+not+...`). Alguns exemplos de cenários onde tabelas de hash podem ser aplicadas incluem servidores de DNS, onde é feita a consulta de endereços IP a partir de nomes de domínio (e.g., chave - `www.google.com`, valor - `142.250.185.14`); e em serviços de encurtamento de URLs, onde os URLs encurtados são mapeados para os seus URLs originais (e.g., chave - `https://tinyurl.com/yfp9jchm`, valor - `https://www.google.com/search?q=how+not+...`).

2.5 Árvores binárias de pesquisa

As árvores binárias, de forma semelhante a listas ligadas, são compostas por nós interligados através de apontadores. No entanto, a organização destes nós difere das listas. Cada nó da árvore binária contém dois apontadores para duas sub-árvores, uma à esquerda e outra à direita do nó (ver Figura 5).

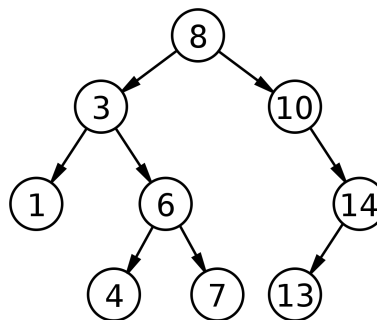


Figura 5: Arquitetura de uma árvore binária de pesquisa

Fonte: wikipedia

Uma árvore binária de pesquisa é ordenada de tal forma que os elementos que são menores do que um nó estarão todos presentes na sua sub-árvore à esquerda, enquanto que todos os elementos maiores do que o nó estarão posicionados na sub-árvore à direita. A árvore tem início num nó especial, ao qual se dá o nome de raiz da árvore.

As árvores binárias de pesquisa são eficientes na pesquisa de conjuntos de valores, quando a condição de pesquisa está associada à ordenação da árvore. Por exemplo, se quisermos encontrar todos os nós, numa árvore de inteiros, com valor inferior a 8, basta-nos encontrar o primeiro nó com valor inferior ou igual a 8, sendo que os restantes valores pretendidos estarão compreendidos na sub-árvore à sua esquerda.

Para garantir a eficiência de operações de pesquisa, a árvore deverá ser balanceada (AVL), de forma a que a diferença entre as alturas das subárvores de um qualquer nó seja no máximo de uma unidade. Para isso, é necessário efetuar operações de computação extra ao inserir um nó na árvore. A **complexidade de todas as operações** consideradas, para esta estrutura, é **logarítmica**.

Em termos de memória, as árvores binárias incorrem um consumo semelhante ao de uma lista ligada com dois apontadores por nó.

Árvores binárias são populares em índices de pesquisa, uma vez que permitem indexar dados de forma a rapidamente encontrar não só valores específicos, mas também valores que obedeçam a relações do tipo "maior que", "menor que", entre outros.

Nota sobre ordenação Diferentes estruturas de dados, como arrays, listas ligadas, ou árvores binárias, podem ser ordenadas. Como vimos para a árvore binária, esta ordenação pode trazer benefícios, por exemplo, ao nível de pesquisa nas estruturas de dados, no entanto incorrem um custo de computação adicional para manter a ordenação da estrutura quando elementos são inseridos ou removidos. Assim, a escolha de ordenar ou não uma estrutura de dados dependerá do caso de uso.

Estrutura	Inserção	Remoção	Acesso	Pesquisa
Array	Linear	Linear	Constante	Linear
Lista ligada	Constante	Constante	Linear	Linear
Tabela de hash	Constante	Constante	N/A	Constante
Árvore binária de pesquisa	Logarítmica	Logarítmica	Logarítmico	Logarítmica

Tabela 1: Complexidade de operações em estruturas de dados¹

¹Para tabelas de hash, acesso por índice não se aplica, pois o acesso é feito por chave, o que corresponde à operação de pesquisa.